

Technik Programista

Zaawansowane aplikacje webowe

Flask – dynamiczne ścieżki

Parametry w URL, konwertery, query string

<imie>, <int:id>, request.args, url_for, redirect, abort(404)

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. Po co dynamiczne ścieżki	3
2. Parametr w URL.....	3
2.1 Kilka parametrów	3
3. Konwertery	4
4. Kalkulator w URL.....	4
4.1 Gdzie tu jest MVC?	4
5. Query string: ?imie=Ola	5
5.1 Parametr w ścieżce czy w query string?	5
6. url_for, redirect, abort	5
6.1 url_for – nie wpisuj adresów ręcznie.....	5
6.2 redirect – przekierowanie.....	5
6.3 abort – zakończ z błędem.....	6
7. Zadania	7
Zadanie 1: Powitanie	7
Zadanie 2: Kalkulator.....	7
Zadanie 3: Tabliczka mnożenia	7
Zadanie 4: Query string	7
Zadanie 5: Mini-baza w słowniku	7
Zadanie 6: Przekierowanie	7
Zadanie 7: Repo	7

1. Po co dynamiczne ścieżki

Sklep ma 5000 produktów. Nie napiszesz 5000 funkcji `@app.route("/produkt/1")`, `/produkt/2...` Potrzebujesz **jednej** trasy, która obsłuży dowolny numer: `/produkt/<id>`. Fragment w nawiasach ostrych to **parametr** – Flask wyciąga go z adresu i przekazuje do funkcji jako argument.

Kontynuujemy w projekcie `flask-start` (venv aktywny, `python app.py`).

2. Parametr w URL

```
@app.route("/czesc/<imie>")
def czesc(imie):
    return f"Cześć, {imie}!"
```

Wejdź na `http://localhost:5000/czesc/Ania` → „Cześć, Ania!”. Na `/czesc/Bartek` → „Cześć, Bartek!”.

Trzy rzeczy muszą się zgadzać:

1. W trasie: `<imie>` – nazwa parametru w nawiasach ostrych.
2. W funkcji: `def czesc(imie)` – argument o **tej samej nazwie**.
3. W ciele: używasz `imie` jak zwykłej zmiennej.

⚠ **Jeśli nazwa w `<>` i w `def` się różni, dostaniesz `TypeError: czesc() got an unexpected keyword argument`. To najczęstszy błąd na tym etapie.**

2.1 Kilka parametrów

```
@app.route("/uzytkownik/<imie>/<nazwisko>")
def uzytkownik(imie, nazwisko):
    return f"Użytkownik: {imie} {nazwisko}"
```

3. Konwertery

Domyślnie parametr jest **stringiem**. `/produkt/5` da `"5"` (tekst), nie `5` (liczba). Żeby dostać liczbę, podajesz typ przed nazwą:

Zapis	Typ w funkcji	Pasuje do	Nie pasuje do
<code><nazwa></code>	str	<code>/x/abc</code> , <code>/x/12</code>	<code>/x/a/b</code> (slash kończy)
<code><int:id></code>	int	<code>/x/12</code>	<code>/x/abc</code> , <code>/x/1.5</code> → 404
<code><float:cena></code>	float	<code>/x/9.99</code>	<code>/x/9</code> → 404 (musi mieć kropkę)
<code><path:sciezka></code>	str ze slashami	<code>/x/a/b/c</code>	–

```
@app.route("/produkt/<int:id>")
def produkt(id):
    return f"Produkt numer {id}, typ: {type(id).__name__}"
```

Sprawdź `/produkt/5` (działa, typ int) oraz `/produkt/abc` (404 – Flask nie dopasował trasy, bo `abc` to nie int).

💡 *Konwerter to pierwsza linia walidacji. Jeśli id ma być liczbą, użyj int: – nie musisz sprawdzać tego w kodzie.*

4. Kalkulator w URL

```
@app.route("/dodaj/<int:a>/<int:b>")
def dodaj(a, b):
    return f"{a} + {b} = {a + b}"

@app.route("/podziel/<int:a>/<int:b>")
def podziel(a, b):
    if b == 0:
        return "Nie dzielimy przez zero", 400
    return f"{a} / {b} = {a / b}"
```

`/dodaj/2/3` → „2 + 3 = 5”. `/podziel/7/0` → komunikat i kod **400 Bad Request** (żądanie poprawne składniowo, ale bez sensu).

4.1 Gdzie tu jest MVC?

Obliczenie `a + b` siedzi w kontrolerze. Przy dodawaniu to nie boli. Ale gdyby to było „oblicz cenę z rabatem, VAT i kosztem dostawy” – powinno wylądować w **modelu** (osobnej funkcji lub klasie), a kontroler tylko ją woła. Zapamiętaj ten moment, wrócimy do niego przy bazie danych.

5. Query string: ?imie=Ola

Drugi sposób przekazania danych w URL to **query string** – wszystko po znaku `?`, pary **klucz=wartość** oddzielone `&`:

```
http://localhost:5000/powitanie?imie=Ola&godzina=14
```

Trasa jest stała (`/powitanie`), a parametry odczytujesz z `request.args`:

```
from flask import Flask, request

@app.route("/powitanie")
def powitanie():
    imie = request.args.get("imie", "nieznajomy")
    godzina = request.args.get("godzina", type=int)
    if godzina is not None and godzina < 12:
        return f"Dzień dobry, {imie}!"
    return f"Witaj, {imie}!"
```

- `request.args.get("imie", "nieznajomy")` – pobierz `imie`, a jak nie ma, użyj wartości domyślnej.
- `type=int` – od razu zamień na liczbę; jak się nie da, zwróci `None`.

5.1 Parametr w ścieżce czy w query string?

W ścieżce <code>/produkt/5</code>	W query string <code>/produkty?kat=laptopy&sort=cena</code>
identyfikuje konkretny zasób	filtruje / sortuje / stronicuje listę
obowiązkowy	opcjonalny
<code>/produkt/5</code> , <code>/uzytkownik/ania</code>	<code>?strona=2</code> , <code>?q=laptop</code> , <code>?sort=cena</code>

💡 Ta sama zasada obowiązuje w Django, ASP.NET i w API dla Reacta.
Wyszukiwarka Google: [google.com/search?q=flask](https://www.google.com/search?q=flask) – to query string.

6. url_for, redirect, abort

6.1 url_for – nie wpisuj adresów ręcznie

`url_for("nazwa_funkcji", parametr=wartość)` buduje URL z nazwy funkcji. Jeśli kiedyś zmienisz trasę z `/produkt/<int:id>` na `/towar/<int:id>`, wszystkie linki zbudowane przez `url_for` zmienią się same.

```
from flask import url_for

@app.route("/linki")
def linki():
    return url_for("produkt", id=5)    # zwróci: /produkt/5
```

6.2 redirect – przekierowanie

```
from flask import redirect
```

```
@app.route("/stary-adres")
def stary():
    return redirect(url_for("index"))
```

Przeglądarka dostaje kod **302** i sama przechodzi na nowy adres. Zobaczysz to w zakładce Network jako dwa żądania.

6.3 abort – zakończ z błędem

```
from flask import abort

PRODUKTY = {1: "Laptop", 2: "Mysz", 3: "Klawiatura"}

@app.route("/produkt/<int:id>")
def produkt(id):
    if id not in PRODUKTY:
        abort(404)
    return f"Produkt: {PRODUKTY[id]}"
```

Słownik **PRODUKTY** to na razie nasza „baza danych”. To jest załączek **modelu** – dane trzymane osobno od kontrolera. W materiale 07 zamienimy słownik na prawdziwą bazę.

7. Zadania

Zadanie 1: Powitanie

Trasa `/czesc/<imie>` zwraca „Cześć, IMIE!”. Trasa `/czesc/<imie>/<int:wiek>` zwraca „Cześć, IMIE, masz WIEK lat”. Sprawdź, co się stanie, gdy w wieku podasz tekst.

Zadanie 2: Kalkulator

Trasy `/dodaj`, `/odejmij`, `/pomnoz`, `/podziel` – każda z `<int:a>/<int:b>`. Dzielenie przez zero → komunikat i kod 400. Dodatkowo `/potega/<int:a>/<int:b>`.

Zadanie 3: Tabliczka mnożenia

Trasa `/tabliczka/<int:n>` zwraca tabliczkę mnożenia dla `n` (od 1 do 10), każdy wiersz w nowej linii. Ogranicz `n` do przedziału 1–20; poza nim → 400.

Zadanie 4: Query string

Trasa `/produkty` obsługuje `?kat=` (kategoria) i `?sort=` (cena / nazwa). Zwraca tekst „Kategoria: X, sortowanie: Y”. Bez parametrów: „wszystkie, domyślne”.

Zadanie 5: Mini-baza w słowniku

Załącz słownik z 5 elementami z Twojej domeny (produkty / ogłoszenia / sale / posty). Trasa `/element/<int:id>` zwraca nazwę albo 404 przez `abort`. Trasa `/elementy` zwraca listę wszystkich, po jednym w linii.

Zadanie 6: Przekierowanie

Trasa `/start` przekierowuje na `/` przez `redirect(url_for(...))`. W zakładce Network pokaż nauczycielowi kod 302 i drugie żądanie.

Zadanie 7: Repo

Commit `Flask: dynamiczne trasy, konwertery, query string`, push. README uzupełnij o listę tras z przykładami adresów.